

GenAI Erfahrungsbericht G4T1

Dieser Bericht dokumentiert den Einsatz von generativer KI im Rahmen des G4T1- Projekts. Verwendete GenAI-Tools waren ChatGPT und Claude, insbesondere für Code- Reviews und Refactoring-Vorschläge.

Code-Reviews

Beispiel: ProductController – Exception Handling

Prompt:

„Analysiere den ProductController auf potenzielle Verbesserungen im Exception Handling. Identifiziere Code-Duplikationen und schlage eine konsistente Fehlerbehandlung vor.“

Ergebnis der Analyse:

Die Analyse ergab mehrere Verbesserungspotenziale. Es wurden wiederholte try- catch-Blöcke mit identischem Exception-Handling identifiziert. Zudem wurde festgestellt, dass eine sehr generische Exception-Behandlung ohne spezifische Fehlertypen verwendet wurde. Außerdem wurde eine ResponseStatusException gefangen und erneut geworfen, was als redundant bewertet wurde.

Vorgeschlagene Lösung:

Die KI schlug vor, das Exception-Handling in eine zentrale Klasse mit `@RestControllerAdvice` auszulagern.

```
@RestControllerAdvice
public class ProductControllerAdvice {

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<ErrorResponse> handleResponseStatusException(
        ResponseStatusException ex) {
        return ResponseEntity.status(ex.getStatusCode())
            .body(new ErrorResponse(ex.getReason()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(
        Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("Unexpected error: " + ex.
                getMessage()));
    }
}
```

Umsetzung:

Die Controller-Methoden wurden vereinfacht und das Exception-Handling vollständig in einen zentralen `@RestControllerAdvice` ausgelagert.

Erweiterte Analyse: HTTP-Fehlerpropagierung

Prompt

„Beim Senden einer ungültigen HTTP-Anfrage aus dem Frontend (FE) scheint der korrekte Code im Backend ausgeführt zu werden, jedoch wird der HTTP-Fehler nicht an das Frontend weitergeleitet. Woran könnte das liegen?“

Lösung

Die Analyse ergab, dass das Problem nicht im fachlichen Code selbst liegt, sondern in der Konfiguration der Sicherheits- und Fehlerbehandlungsschicht.

In Spring Boot werden HTTP-Fehler intern häufig an den Endpunkt `/error` weitergeleitet. Ist dieser Endpunkt durch Spring Security nicht freigegeben, kann der eigentliche HTTP-Statuscode nicht korrekt an das Frontend propagiert werden.

Wesentlicher Lösungsansatz: Der Endpunkt `/error` muss explizit für alle Benutzer freigegeben werden.

```
http.authorizeHttpRequests()
    .requestMatchers("/error").permitAll()
    .anyRequest().authenticated();
```

Nach dieser Anpassung konnten HTTP-Fehler wieder korrekt an das Frontend zurückgegeben werden.

Testfallideen

Beispiel: CartService – Edge Cases

Verwendeter Prompt:

„Generiere Testfälle für den CartService, insbesondere für Edge Cases und Fehlerzenarien. Berücksichtige Transaktionssicherheit und Race Conditions.“

Beispiel für einen Testfall:

```
@Test
@Transactional
void testAddItemToCart_InsufficientStock() {
    Product product = createProductWithStock(5);
    CartItem cartItem = new CartItem();
    cartItem.setProductId(product.getId());
    cartItem.setQuantity(10);

    assertThrows(ResponseStatusException.class, () -> {
        cartService.addItemToCart(cartItem);
    });

    Cart cart = cartService.getCart();
    assertTrue(cart.getItems().isEmpty());
}
```

Umsetzung:

Mehr als 7 Testfälle wurden auf Basis dieser Vorschläge implementiert.

Fehleranalyse im Backend

Prompt

„Ich kann die Ursache dieses Fehlers in meinem Spring-Backend nicht lokalisieren. Kannst du mir sagen, wo dieser Fehler vermutlich entsteht?“

Error-Info

Hinweis: Der folgende Stacktrace ist vollständig frei erfunden und dient nur zur Illustration.

```
org.springframework.dao.InvalidDataAccessApiUsageException:
failed to lazily initialize a collection of role:
com.example.demo.domain.Order.items,
could not initialize proxy - no Session

    at org.springframework.orm.jpa.EntityManagerFactoryUtils
        .convertJpaAccessExceptionIfPossible(
            EntityManagerFactoryUtils.java:371)
    at org.springframework.orm.jpa.AbstractEntityManagerFactoryBean
        .translateExceptionIfPossible(
            AbstractEntityManagerFactoryBean.java:551)
    at org.springframework.dao.support.
        ChainedPersistenceExceptionTranslator
            .translateExceptionIfPossible(
                ChainedPersistenceExceptionTranslator.java:61)
    at org.springframework.context.event.
        SimpleApplicationEventMulticaster
            .invokeListener(SimpleApplicationEventMulticaster.java:178)
```

Lösung

Die Analyse zeigt, dass der Fehler aus dem Datenbankzugriff stammt. Konkret wird auf eine lazy-geladene Entität außerhalb eines aktiven Persistenzkontexts zugegriffen.

Dies tritt häufig in Kombination mit asynchronen Event-Listnern auf, da diese nicht denselben Hibernate-Session-Kontext wie der ursprüngliche Request verwenden.

Der Fehler ist somit auf eine Kombination aus:

- Lazy Loading von JPA-Entitäten
- asynchronen Listnern
- und fehlenden Transaktionsgrenzen

zurückzuführen.

Beispieldaten: Review-Seedings

Prompt

„Bitte ergänze ein paar Beispieldaten in folgender Form:

```
INSERT INTO REVIEW (SCORE, PRODUCT_ID, TIMESTAMP, USER_ID, CONTENT) VALUES
(...);
```

Beachte dabei, dass jeder User nur ein Review pro Produkt schreiben darf. Es gibt noch user1 und elvis.“

Antwort

Im Folgenden werden Beispieldaten für Reviews zum Produkt DDR5 32GB ergänzt. Dabei wird sichergestellt, dass jeder Benutzer maximal ein Review für dieses Produkt erstellt (*ein Review pro User und Produkt*).

```
-- add comments
-- Reviews for DDR5 32GB

INSERT INTO REVIEW (SCORE, PRODUCT_ID, TIMESTAMP, USER_ID, CONTENT)
VALUES (
    5.0,
    (SELECT ID FROM PRODUCT WHERE NAME = 'DDR5_32GB'),
    CURRENT_TIMESTAMP,
    (SELECT ID FROM USERX WHERE USERNAME = 'adriano'),
    'A small price to pay for RAM.'
);

INSERT INTO REVIEW (SCORE, PRODUCT_ID, TIMESTAMP, USER_ID, CONTENT)
VALUES (
    3.0,
    (SELECT ID FROM PRODUCT WHERE NAME = 'DDR5_32GB'),
    CURRENT_TIMESTAMP,
    (SELECT ID FROM USERX WHERE USERNAME = 'user2'),
    'This is the most expensive part of my whole build...'
);

INSERT INTO REVIEW (SCORE, PRODUCT_ID, TIMESTAMP, USER_ID, CONTENT)
VALUES (
    4.0,
    (SELECT ID FROM PRODUCT WHERE NAME = 'DDR5_32GB'),
    CURRENT_TIMESTAMP,
    (SELECT ID FROM USERX WHERE USERNAME = 'user1'),
    'Stable performance, easy XMP setup, and no issues under load.'
);

INSERT INTO REVIEW (SCORE, PRODUCT_ID, TIMESTAMP, USER_ID, CONTENT)
VALUES (
    2.0,
    (SELECT ID FROM PRODUCT WHERE NAME = 'DDR5_32GB'),
    CURRENT_TIMESTAMP,
    (SELECT ID FROM USERX WHERE USERNAME = 'elvis'),
    'Works fine, but the price/performance ratio hurts right now.'
);
```

Zusammenfassung und Erkenntnisse

Der Einsatz von GenAI führte zu einer deutlichen Zeitersparnis bei der Identifikation von Code-Duplikationen und potenziellen Problemen. Zudem konnte die Code-Qualität durch systematische Analysen von Edge Cases und Fehlerszenarien verbessert werden.

Herausforderungen zeigten sich insbesondere beim begrenzten Kontextverständnis der KI, wodurch teilweise mehrere Iterationen notwendig waren. Außerdem mussten alle Vorschläge kritisch überprüft werden, da manche Ansätze für das Projekt zu komplex gewesen wären.